

PRODUCT REQUIREMENTS DOCUMENT (PRD)

AI-Powered Conversational Chatbot

Project Documentation

Document	Product Requirements Document
Project	AI-Powered Chatbot
Version	1.0
Status	Project / Academic Prototype
Primary Goal	Provide an interactive conversational interface using NLP/AI processing and backend APIs.

1. Executive Summary

The AI-Powered Conversational Chatbot is a web-based application designed to allow users to communicate with an intelligent chatbot through a simple, responsive chat interface. The system accepts natural-language questions, sends them through a backend API and NLP/AI processing layer, and returns an appropriate conversational response. The product focuses on usability, responsive design, reliable request handling, conversation continuity, and a modular architecture that can be extended with additional AI capabilities.

2. Product Vision

Build a simple and accessible conversational assistant that demonstrates how a modern frontend, backend APIs, and NLP/AI processing can work together to provide an interactive user experience.

3. Problem Statement

Users often need a quick way to ask questions and receive information without navigating multiple interfaces. The project addresses this by providing a centralized conversational interface where users can submit natural-language queries and receive automated responses.

4. Objectives

- Enable natural-language interaction between users and the chatbot.
- Process user queries through NLP/AI logic.
- Provide responses through backend API integration.
- Maintain a clear and interactive conversation view.
- Support responsive use across desktop, tablet, and mobile devices.
- Handle invalid input, failed requests, and server/API errors gracefully.
- Keep the architecture modular enough for future AI and product features.

5. Target Users

- Students and academic users demonstrating an AI/NLP application.
- General users seeking quick conversational assistance.
- Developers evaluating frontend-backend-AI integration concepts.

6. Scope

In Scope

- Chat interface and message exchange.
- Natural-language query processing.
- Backend API communication.
- AI/NLP response generation or retrieval.
- Conversation display/history within the active session.
- Loading and response states.
- Input validation and error handling.
- Responsive user interface.

Out of Scope for the base version

- Voice conversation, image understanding, autonomous agents, payment features, and enterprise-grade identity management unless separately implemented.
- Long-term user memory across accounts is considered a future enhancement.

7. Functional Requirements

7.1 Chat & Conversation

- User can enter a natural-language message.
- User can submit a message using the send action.
- User and chatbot messages are visually distinct.
- Messages appear in chronological order.
- The interface supports an ongoing conversational flow.
- Conversation content can be retained for the active session.

7.2 NLP / AI Processing

- System receives the user's text query.
- Query is passed to the NLP/AI processing layer.
- The system interprets the query and generates or retrieves a suitable response.
- The response is returned to the frontend through the backend/API layer.

7.3 Backend API

- Expose an API endpoint for chat/query processing.
- Validate incoming request data.
- Forward the query to the configured AI/NLP logic.

- Return a structured response to the frontend.
- Return meaningful error responses when processing fails.

7.4 UI States

- Idle state.
- Message-sending state.
- Loading / waiting-for-response state.
- Successful response state.
- Validation-error state.
- API/server-error state.

7.5 Input Validation

- Prevent empty messages from being submitted.
- Handle excessively long or malformed input according to configured limits.
- Sanitize/validate API request data.

8. User Experience Requirements

- Clean, minimal, chat-focused interface.
- Clear distinction between user and chatbot messages.
- Obvious message input and send controls.
- Readable typography and accessible spacing.
- Responsive layout for common screen sizes.
- Feedback while the system is processing a query.
- Clear error messages without exposing technical internals.

9. Technical Requirements

- Frontend: React-based web interface.
- Runtime/package tooling: Node.js and npm.
- Backend: API-based architecture suitable for chatbot request processing.
- AI/NLP layer: configurable NLP/AI processing component or service.
- Communication: HTTP/HTTPS API requests between frontend and backend.
- Development environment: VS Code or equivalent IDE.
- Architecture should keep presentation, API, and AI-processing concerns logically separated.

10. System Workflow

Primary flow: User → Chat UI → Frontend validation → Backend API → NLP/AI processing → Response → Frontend → Conversation display.

Detailed flow

- User opens the chatbot application.
- User enters a question/message.
- Frontend validates the input.
- Frontend sends the request to the backend API.
- Backend validates and processes the request.
- NLP/AI component interprets the query and prepares a response.
- Backend returns the response.
- Frontend updates the conversation view.
- If an error occurs, the frontend shows a user-friendly error state.

11. Non-Functional Requirements

- Performance: responses and UI interactions should feel responsive under normal project loads.
- Availability: the UI should remain usable when individual API requests fail.
- Scalability: API and AI layers should be replaceable or extendable.
- Maintainability: use modular components and clear separation of concerns.
- Usability: common chat actions should require minimal learning.
- Compatibility: support modern desktop and mobile browsers.
- Reliability: prevent duplicate/invalid submissions where practical.

12. Error Handling

- Empty input → prompt the user to enter a message.
- Invalid request → return validation feedback.
- AI/API timeout → show a retry-friendly message.
- Server failure → show a generic service-unavailable message.
- Unexpected response → safely handle the response without breaking the UI.
- Network failure → inform the user that the request could not be completed.

13. Security & Privacy Considerations

- Use HTTPS in production.
- Do not expose private API keys in frontend code.
- Validate and sanitize user input on the server side.
- Apply rate limiting if the application is publicly deployed.
- Avoid logging sensitive user content unnecessarily.
- Keep secrets in environment variables or a secure secret-management mechanism.

14. Testing Requirements

- UI test: message input and send interaction.

- Validation test: empty message rejection.
- API test: valid request returns expected response structure.
- Error test: API failure produces a safe user-facing message.
- Responsive test: verify layout across desktop and mobile widths.
- Conversation test: multiple messages appear in correct chronological order.
- Integration test: frontend → backend → AI/NLP → frontend response flow.

15. Future Enhancements

- User authentication and personalized accounts.
- Persistent conversation history stored in a database.
- Multiple chat sessions and chat renaming.
- Voice input and text-to-speech.
- File/document-based question answering.
- Multilingual chatbot support.
- Context-aware long conversations.
- Admin dashboard and analytics.
- Feedback/rating mechanism for responses.
- Integration with external knowledge bases and additional APIs.

16. Success Criteria

- Users can successfully send natural-language queries.
- The system returns an appropriate chatbot response through the API flow.
- Conversation messages are displayed clearly and consistently.
- Invalid input and service failures do not crash the application.
- The interface works across common desktop and mobile screen sizes.
- The codebase remains modular enough to add future features.

17. Feature Summary

Module	Key Features
Chat Interface	Message input, send action, user/bot message display, conversation flow
AI/NLP	Natural-language query processing, response generation/retrieval
Backend	API endpoint, request validation, AI integration, structured responses
UX	Responsive UI, loading states, clear errors, readable conversation
Reliability	Validation, network/API error handling, safe failure states
Architecture	Separated frontend, backend API, and AI/NLP layers
Future-ready	Persistent history, authentication, voice, multilingual, document Q&A

Note: This PRD documents the project scope and feature set based on the chatbot project description available in the conversation. It intentionally distinguishes the base project from optional future enhancements.